

Git



Introdução.....	3
Comandos Básicos.....	5
Boas Práticas.....	8
Exemplo de Fluxo de Trabalho com Git.....	10

Introdução

O que é GitHub?

O GitHub é uma plataforma online que hospeda repositórios Git, ou seja, armazena e permite gerenciar projetos de software com controle de versão.

O que é Git?

É um sistema de controle de versão distribuído, um software de código aberto, usado para rastrear e gerenciar as mudanças em arquivos ao longo do tempo.

Git permite que você controle e acompanhe o histórico de modificações em projetos, especialmente em desenvolvimento de software, permitindo que você volte a versões anteriores, compare alterações e trabalhe em equipe de forma colaborativa.

Por que Usar Git?

Imagine que você está desenvolvendo um projeto e agora seu amigo também quer participar do desenvolvimento. Vocês podem usar o drive e ter que mesclar os códigos manualmente, ou aguardar um terminar o que está fazendo para o outro trabalhar, estressante e perde muito tempo. Mas com o Git podem fazer algo melhor, usando GitHub como plataforma vocês vão usar essa ferramenta para desenvolver de forma colaborativa, como? Simples, o Git permite que duas ou mais pessoas desenvolvam o mesmo projeto sem que uma tenha que esperar a outra terminar, mas cuidado, não é tão simples assim. Por exemplo, duas pessoas não podem modificar o mesmo código ao mesmo tempo, imagine um código simples em python:

Modificação de Maria:

```
print("Hello World!!!")
```

Modificação de Pedro:

```
print("Olá Mundo!!!")
```

Quando eles enviarem, o Git vai olhar para as duas atualizações e não vai saber qual manter, então surgirá o famoso e temido Conflito de Merge! Mas calma, dá para consertar, no GitHub irá mostrar os arquivos em conflito, basta deixar o código que quiser, ou até mesmo os dois, então é só fazer o merge.

Instalar Git

Primeiramente você precisará baixar e instalar o git em sua máquina

<https://git-scm.com/>

Para instalar é bem simples, apenas clicar nos botões de 'next'

Criar Repositório do Zero

Entendendo o Origin

Origin nada mais é do que uma variável que guarda a URL do repositório remoto, você pode até usar outro nome, mas por padrão é origin

Adicionar a Remote

Após criar o repositório no GitHub, use os comandos a seguir para continuar o desenvolvimento

Adicionar a remote no repositório local

```
git remote add origin URL
```

Mudar Nome da Branch Padrão

Mudar o nome da branch padrão de master para main (ou outro nome desejado)

```
git branch -M main
```

Você agora tem um repositório configurado localmente, você pode fazer push e o que mais for necessário

Comandos Básicos

Inicializar o Repositório Local

Para inicializar um repositório git local

```
git init
```

Clonar um Repositório

Para colaborar com um projeto em desenvolvimento, partindo do pressuposto que você tem acesso ao repositório desejado, basta cloná-lo, isso trará as configurações de rotas e remotes certas sem retrabalho

```
git clone URL
```

Criar Branch

Para criar uma branch local use

```
git checkout -b nome/branch/nova
```

Não usar caracteres especiais

Listar Branches

Talvez seja necessário listar quais as branches existentes localmente, para isso use

```
git branch
```

Deletar Branches

Você pode optar por deletar branches que criou, normalmente após um push para uma nova branch há o costume de removê-la das branches locais

```
git branch -d nome/branch
```

Mudar de Branch

Para mudar de branch em que está trabalhando

```
git checkout nome/branch
```

Verificar Status

Caso precise verificar qual o status atual de seu repositório local quando comparado ao remoto use

```
git status
```

O que for retornado em vermelho é porque são modificações que não foram adicionadas a um commit, em verde são alterações adicionadas em um commit. Se aparecer a frase “Already up to date” significa, na maioria das vezes, que está tudo em ordem, nada para enviar ao repositório remoto

Adicionar Arquivos para Commit

Para adicionar arquivos para um commit existem algumas formas, caso queira adicionar todas as modificações em um commit

```
git add .
```

Para adicionar arquivos separados

```
git add arquivo1 arquivo2
```

Mensagem de Commit

É obrigatório escrever uma mensagem de commit, isso é executado após adicionar arquivos para commitar

```
git commit -m "sua mensagem"
```

Dentro da mensagem de commit você pode usar caracteres especiais

Enviar Alterações Locais para Repositório Remoto

Quando um processo de desenvolvimento é terminado, seja funcionalidade ou correção, essas alterações devem ser enviadas para o repositório remoto

```
git push
```

No caso de ser o primeiro push naquela branch usa-se

```
git push -u origin nome/branch
```

Buscar Alterações do Repositório Remoto para o Local

Para pegar as alterações e atualizações que foram enviadas para o repositório remoto usa-se

```
git pull
```

É importante ter atenção que, caso não tenha enviado suas alterações e você force um pull, suas alterações serão sobrescritas, ou seja, perdidas

Boas Práticas

Conventional Commits

Para evitar problemas com mensagens de commits genéricas como “Aquele funcionalidade lá” ou “Alteração de fulano”, criou-se o Conventional Commits, que é uma forma de criar as mensagens usadas no commit

`tipo(escopo): título`
`Descrição`

Tipo (obrigatório): Indica o propósito da mudança

- feat: Adição de uma nova funcionalidade
- fix: Correção de um bug
- docs: Mudanças apenas na documentação
- style: Alterações que não afetam a lógica do código (ex.: formatação, espaçamento)
- refactor: Alteração de código que não corrige bugs nem adiciona funcionalidades
- test: Adiciona ou corrige testes
- chore: Atualizações de ferramentas, dependências ou configurações

Escopo (opcional): Qual parte da aplicação foi afetada

- Header, Footer, Sign In, Sign Out, etc

Título: Um título claro e conciso da mudança

Descrição (obrigatória): Descreve as alterações realizadas

Exemplo de mensagem de commit:

`feat(auth): adiciona formulário de login`

Inclui um formulário de login no componente
`Login.jsx`.

O formulário possui validação básica de e-mail e senha.

Método de Versionamento de Código

Um padrão muito usado é o *SemVer* (Versionamento Semântico)

MAJOR.MINOR.PATCH

MAJOR: modificação que quebra compatibilidade com versões anteriores

MINOR: feature, funcionalidades novas que mantém compatibilidade com versões anteriores

PATCH: fix, correções de bugs etc

Exemplo:

`v1.2.9`

Pull Requests

Sempre gere os famosos PRs (Pull Request). Basicamente é uma forma de manter o código principal longe de problemas e conflitos. Imagine que você está desenvolvendo, é o fim do expediente e você vai fazer um belo de um commit para ir dormir. Você manda o commit direto para o código principal (branch principal), no dia seguinte você acorda e vai testar e vê que algo aconteceu e tudo parou de funcionar, pois é, você enviou um código com bug. Se você tivesse gerado a PR isso não aconteceria.

Vamos voltar no dia anterior, vamos seguir os mesmos passos, mas desta vez você criou uma branch nova, agora fez um commit e criou um PR, pode dormir de consciência limpa. No dia seguinte você acorda e testa o código que está no PR mas que não foi mergeado. Bug, mas calma, seu código principal ainda funciona, é só consertar e mergear.

Exemplo de Fluxo de Trabalho com Git

Com uma funcionalidade desenvolvida, as alterações devem ir para o git.

Primeiramente, deverá criar uma branch, isso gerará um PR quando enviar ao git

```
git checkout -b feat/auth
```

Verificar status (pode verificar isso quando quiser)

```
git status
```

Adicionar as alterações na stage da nova branch

```
git add .
```

Escrever mensagem de commit

```
git commit -m "feat(auth): adiciona formulário de login"
```

```
Inclui um formulário de login no componente Login.jsx.
```

```
O formulário possui validação básica de e-mail e senha."
```

Enviar para o repositório remoto

```
git push -u origin feat/auth
```

Voltar para branch principal

```
git checkout main
```

Deletar a branch criada para gerar PR

```
git branch -d feat/auth
```

Atualizar repositório local

```
git pull
```